Comparativo de conformidade — pix-mocked-server × API Pix (BACEN)

Pendências da implementação em relação à especificação oficial

2026-11-09

Sumário

1. Prompt utilizado **1**

2. Escopo e metodologia **1**

3. Resumo executivo **2**

4. Endpoints ausentes **2**

5. Bugs e desvios encontrados **3**
5.1 `/cobr`: controller não registrado (bloqueante) 3
5.2 `src/controllers/cobr.controller.ts` 3
5.3 Rotas GET `:txid` duplicadas (código morto) 3
5.4 Status HTTP de POST `/cob` 3

6. Autenticação **4**

7. Itens conformes **4**

8. Recomendações priorizadas **4**

Apêndice A: verificação em execução **5**

Apêndice B: regeneração deste PDF **5**

1. Prompt utilizado

> Devin comparar se há pendências dessa implementação em relação ao que é especificado pelo BACEN. Gere um print ou PDF do resultado, incluindo o prompt usado.

2. Escopo e metodologia

- **Implementação analisada:** `ntt-sandrada/pix-mocked-server`, branch `main` (commit `3b06a4c`).
- **Especificação de referência:** API Pix do BACEN, https://bacen.github.io/pix-api/, `openapi.yaml` versão **2.11.0-rc1** (https://github.com/bacen/pix-api).
- **Como foi feito:**
  1. Leitura estática dos controllers em `src/controllers/`, do `src/app.module.ts`, do `src/http.interceptor.ts` e do serviço `src/core/app/psp.ts`.

2. Extração da lista de paths/métodos do `openapi.yaml` oficial.
3. Execução local do servidor (`src/main.ts`: API na porta 3000, autenticação na 3001) e chamadas `curl` para confirmar os status HTTP de cada rota.

3. Resumo executivo

Grupo	Situação
`/cob`	Conforme
`/cobv`	Conforme
`/loc`	Conforme (inclui validação de filtros)
`/cobr`	**Não conforme**: controller não registrado e com bugs
`/rec`	**Parcial**: só `POST /rec`
`/pix` (consulta e devolução)	**Ausente**
`/webhook`	**Ausente** (`webhook.controller.ts` está vazio)
`/lotecobv`	**Ausente**
Autenticação OAuth2	**Não conforme** (formato do token e esquema de header)

4. Endpoints ausentes

Status confirmado em execução local com header de autenticação válido (`mocked-token: 123-abc`): todas as rotas abaixo retornam **404** `Cannot <MÉTODO> <rota>`.

Grupo BACEN	Métodos e paths na especificação	Implementação
Pix	`GET /pix`, `GET /pix/{e2eid}`	Ausente
Devolução	`PUT /pix/{e2eid}/devolucao/{id}`, `GET /pix/{e2eid}/devolucao/{id}`	Ausente
Webhook	`PUT/GET/DELETE /webhook/{chave}`, `GET /webhook`	Ausente (controller vazio, 0 bytes)
Lote CobV	`PUT/PATCH/GET /lotecobv/{id}`, `GET /lotecobv`	Ausente
Recorrência	`GET /rec/{idRec}`, `PATCH /rec/{idRec}`, `GET /rec` (lista), desistência	Parcial: só `POST /rec` existe

Arquivos relacionados:

- `src/controllers/webhook.controller.ts`: vazio;
- `src/controllers/rec.controller.ts`: somente `@Post()` (linha 13).

Observação sobre "desistência": na versão 2.11.0-rc1 da especificação não existe rota dedicada de desistência em `/rec`. O cancelamento de uma recorrência é feito via `PATCH /rec/{idRec}` (status `CANCELADA`), que também não existe na implementação.

Outros grupos da especificação que também não estão implementados (fora do escopo principal, listados para completude): `/locrec`, `/solicrec`, `/webhookrec`, `/webhookcobr`, `/webhookcob`, `/webhookcobv`, `GET /{pixUrlAccessToken}`, `GET /cobv/{pixUrlAccessToken}` e `GET /rec/{recUrlAccessToken}`.

5. Bugs e desvios encontrados

5.1 `/cobr`: controller não registrado (bloqueante)

`CobRController` **não está na lista** `controllers` de `src/app.module.ts` (linha 10: `[CobController, CobVController, LocationController, RecController]`). Por isso **todas** as rotas `/cobr` retornam 404 em execução, inclusive `POST /cobr`. Além disso, `PSP.addCobR` (`src/core/app/psp.ts`, linhas 100–104) é um stub que retorna `null`. Mesmo com o controller registrado, `POST /cobr` não criaria a cobrança.

5.2 `src/controllers/cobr.controller.ts`

Linha	Problema
17	`PUT /cobr/:txid` retorna `await null` (a chamada `this.pspService.addCob(itm)` está comentada). Pela especificação, deve criar a cobrança e responder 201.
28–33	`POST /cobr/:txid/retentativa/:data` ignora o parâmetro `:data` (não há `@Param("data")`). Também recebe um body `CobRPutRequest` que a especificação não define e chama `addCobR` como se fosse uma criação.
39	`read()` delega para `readCob` em vez de um `readCobR`.
46	`list()` delega para `listCob` em vez de um `listCobR`.
53	`edit()` delega para `editCob` em vez de um `editCobR`.
56–59	`remove()` está decorado com `@Get(":txid")` (rota duplicada e inalcançável) e delega para `removeCob` em vez de um `removeCobR`.

> O PSP **não possui** métodos `readCobR`, `listCobR`, `editCobR` nem `removeCobR`. Trocar a delegação exige primeiro implementar esses métodos no serviço, sobre a tabela `COBR`, que já é criada em `src/main.ts`.

Além disso, o mapa de tratamento de erros 400 em `src/http.interceptor.ts` (linhas 68–87) não tem a chave `cobr`. Um erro de validação em `/cobr` resultaria em `handle` indefinido, ou seja, um 500 em vez do 400 `CobROperacaoInvalida`.

5.3 Rotas `GET :txid` duplicadas (código morto)

Os métodos `remove()` estão decorados com `@Get(":txid")`, que já está mapeado pelo `read()`. O NestJS resolve a primeira rota registrada, então `remove()` **nunca é executado**:

- `src/controllers/cob.controller.ts`, linha 49
- `src/controllers/cobv.controller.ts`, linha 40
- `src/controllers/cobr.controller.ts`, linha 56

A especificação BACEN não define `DELETE` para `/cob`, `/cobv` ou `/cobr`. A remoção é feita via `PATCH` com `status = REMOVIDA_PELO_USUARIO_RECEBEDOR`. A recomendação é remover esses métodos (ou movê-los para uso interno), e não convertê-los em `DELETE`.

5.4 Status HTTP de `POST /cob`

A especificação exige **201** para `POST /cob`. A suspeita inicial era que a rota retornasse 200 (não há `@HttpCode(201)` em `src/controllers/cob.controller.ts`, linha 21). **Na verificação em execução, a rota retornou 201**, porque o NestJS usa 201 como status padrão de `@Post()`. Portanto, **está conforme**, e o item foi rebaixado de bug para observação. Adicionar `@HttpCode(201)` explicitamente é opcional, só para deixar a intenção clara.

6. Autenticação

Aspecto	Especificação BACEN	Implementação
Resposta do token	OAuth2 `client_credentials` (RFC 6749): `access_token`, `token_type: "Bearer"`, `expires_in`	`src/controllers/auth.controller.ts`, linha 16: retorna `accesstoken` (sem o underscore) e não retorna `token_type`. Exemplo real: `{"accesstoken":"123-abc","expires_in":300}`
Envio do token na API	`Authorization: Bearer <access_token>` (security scheme OAuth2, com escopos `cob.read`, `cob.write` etc.)	O interceptor (`src/http.interceptor.ts`, linha 47) lê o header proprietário `mocked-token` e injeta internamente o header `IdOwner` (linha 52). Os controllers leem `headers["IdOwner"]`. `Authorization: Bearer ...` resulta em **401**.
Credenciais inválidas	Erro OAuth2 (`invalid_client`, 401)	O serviço lança `Error` genérico, e o cliente recebe **500** `Internal server error`.
Escopos	Escopos por recurso (`cob.write`, `pix.read`, `webhook.write`...)	Não há verificação de escopo.

7. Itens conformes

- **`/cob`**: `PUT /cob/{txid}` (201), `POST /cob` (201), `GET /cob/{txid}` (com `revisao`), `GET /cob` (lista com filtros e paginação) e `PATCH /cob/{txid}`.
- **`/cobv`**: `PUT /cobv/{txid}` (201), `GET /cobv/{txid}`, `GET /cobv` e `PATCH /cobv/{txid}`.
- **`/loc`**: `POST /loc`, `GET /loc` e `GET /loc/{id}`, `DELETE /loc/{id}/txid`, com validação de filtros em `src/controllers/location.controller.ts` (`parseLocFilter`: `inicio`/`fim` obrigatórios, rejeição de datas inexistentes, `txIdPresente`, `tipoCob` e paginação) e erros no formato `PayloadLocationOperacaoInvalida`.
- **`POST /rec`**: implementado e validado (porém o restante de `/rec` está ausente; ver seção 4).
- **Formato de erros**: o interceptor gera problem details com `type` `https://pix.bcb.gov.br/api/v2/error` `title`, `status`, `detail` e `violacoes`, como na especificação.

  Correção em relação ao escopo inicial: `POST /cobr` era considerado conforme, mas **não está**: a rota não é exposta (seção 5.1) e o serviço retorna `null`.

8. Recomendações priorizadas

1. Registrar `CobRController` em `AppModule`, implementar `addCobR`, `readCobR`, `listCobR` e `editCobR` no PSP, corrigir as delegações e o `PUT` do controller, e adicionar `cobr` ao interceptor de erros.
2. Corrigir a resposta do `/auth` para `{ access_token, token_type: "Bearer", expires_in }`, aceitar `Authorization: Bearer` no interceptor (mantendo `mocked-token` por compatibilidade, se desejado) e retornar 401 para credenciais inválidas.
3. Remover os métodos `remove()` mapeados em `@Get(":txid")` duplicado.
4. Implementar `/pix` (consulta e devolução) e `/webhook`, que são os mais usados por integradores.
5. Completar `/rec` (`GET`, `PATCH` e lista) e implementar `/lotecobv`.
6. Avaliar os grupos restantes (`/locrec`, `/solicrec`, `/webhookrec`, `/webhookcobr`, `/webhookcob`, `/webhookcobv`, payload locations).

Apêndice A: verificação em execução

```
$ curl -X <método> -H 'mocked-token: 123-abc' http://localhost:3000<rota>
POST  /cobr                        -> 404 Cannot POST /cobr
GET   /cobr                        -> 404 Cannot GET /cobr
PUT   /cobr/<txid>                 -> 404
GET   /pix | /pix/E123             -> 404
GET   /webhook | PUT /webhook/x    -> 404
PUT   /lotecobv/1                  -> 404
GET   /rec | /rec/R1, PATCH /rec   -> 404
POST  /cob  (payload válido)       -> 201
POST  /cob  (Authorization: Bearer 123-abc) -> 401

$ curl -X POST http://localhost:3001/auth \
    -d '{"grant_type":"client_credentials","client_id":"123","client_secret":"abc"}'
{"accesstoken":"123-abc","expires_in":300}

$ curl -X POST http://localhost:3001/auth \
     -d '{"grant_type":"client_credentials","client_id":"x","client_secret":"y"}'
{"statusCode":500,"message":"Internal server error"}
```

Apêndice B: regeneração deste PDF

```
./docs/gerar-comparativo-bacen.sh   # requer pandoc + xelatex (texlive-xetex)
```